

SecondBrain

The Complete Engineering Record
From an empty folder to a published research contribution

Every idea considered, every direction abandoned, every defect found,
every constraint imposed, and every concept the system is built upon.

Project	SecondBrain — an autonomous personal cognitive knowledge system
Repository	github.com/ALPHA-TF141/Second-Brain-Major-Project
Author	Maria Immanuel L
Programme	B.Tech, Computer Science and Engineering
Institution	Vel Tech Rangarajan Dr. Sagunthala R&D; Institute of Science and Technology
Compiled	22 September 2026
Volume structure	Seven volumes plus six appendices
Source of truth	231 Git commits; every figure in this report was read from the repository or from measured benchmark output

Preamble

This is not a user manual and not a research paper. It is the record of how a piece of software came to be — including the parts that went wrong. It documents the reasoning behind decisions, the alternatives that were rejected, the moments where the plan changed direction, and the theory that underpins each component.

It is written this way deliberately, for three reasons. First, a system of this size cannot be understood from its source code alone: the code shows what was built, never why, and never what was tried first and discarded. Second, an engineering project is judged on judgement rather than on adherence, and judgement is only visible in the deviations. Third, and most practically, the author needed a document that a supervisor or examiner could read and understand the whole system from, without reading 17,000 lines of Python.

Throughout, a distinction is maintained between three things that are easy to confuse. **What was intended** is recorded as stated. **What was built** is recorded as verified. **What was measured** is recorded as measured, with the method named. Where something was not measured, this document says so rather than estimating.

On the honesty of this record

Twenty-seven defects are documented here, including one in which a verification test deleted 123 of the author's genuine memory cards. That incident is recorded in full, with its cause and its fix, because a defect register that omits the worst defect is not a defect register. The same principle applies to the research results: the headline finding is a negative one, and it is reported as such.

Table of Contents

Volume I — Origin and Vision

1. The problem, stated plainly
2. Why the first framing was wrong
3. Personal knowledge management: the theory
4. The reference point, analysed properly
5. The author's brief: every constraint recorded

Volume II — Architecture

6. Design philosophy
7. The seven-stage pipeline
8. The data model in full
9. The agent layer
10. The API surface
11. The desktop shell and interface
12. Technology selection and its trade-offs

Volume III — Build Chronology

Phases 0 through 7, each with the request, the reasoning, and the failures.

Volume IV — Concepts and Theory

Fourteen chapters covering the theory the system is built on.

Volume V — Deviations

Volume VI — Defects

Volume VII — Research Contribution

Volume VIII — Engineering Practice

Appendices

A schema · B API · C file map · D glossary · E timeline · F input log

Volume I — Origin and Vision

1. The problem, stated plainly

The project began with a specific and unglamorous frustration. The author reads technical material constantly: documentation, papers, lecture notes, videos, email, lecture slides. The material was genuinely read — comprehension was not the problem. The problem was that it could not be *recalled* when it later mattered.

A concrete version of the problem: six weeks after reading about a technique, a question arises that the technique would answer. The person *knows* they have read something relevant. They cannot remember where, or what it said exactly. They search for it, fail, and either re-read the whole source or proceed without it. Multiply this by every subject the person studies and the cost is substantial.

The distinction that shaped the entire project

The problem is not **storage**. Storage is free and solved. The problem is not even **search**, in the ordinary sense — a file search would find the document, and a web search would find a hundred copies of it. The problem is **recall under a question**: being able to ask ‘what did I learn about X?’ and receive an answer assembled from everything one has encountered, without having predicted in advance that X would matter. This distinction eliminates most conventional solutions, because all of them require the user to have foreseen the question.

Three properties follow from that framing, and they recur throughout this document as design constraints.

Property	Why it is required	What it rules out
Capture must be automatic	The user cannot decide in advance what will matter, so they cannot be asked to save it.	Manual note-taking, bookmarking, tagging, and any system that begins with an empty page.
What matters must be inferred	Determining relevance is the work the user is bad at; delegating it back to them recreates the original problem.	Folder structures, manual categorisation, and any scheme requiring the user to maintain it.
Questions are natural language	The question is formed at the moment of need, in the user's own words, not as keywords.	Keyword search, filter panels, and query languages.

Table 1 — The three properties that follow from the problem statement, and what each of them eliminates.

2. Why the first framing was wrong

The first design was a note-taking application. It had a text editor, a search box, and a tag system. It would have taken a fortnight to build and would have been abandoned within that fortnight, for a reason worth stating precisely because it is the reason most personal knowledge tools fail.

Such a system asks the user to perform a judgement each time they encounter something: is this worth saving? If so, where does it belong? What should it be tagged with? That judgement is exactly the capability the user lacks — if they could reliably judge what would matter in six weeks, they would not have the recall problem. The system therefore demands the one skill whose absence created the need for it.

THE VICIOUS CIRCLE

user cannot predict what will matter
 ▼
 so cannot curate reliably
 ▼
 so the system fills with noise, or stays empty
 ▼
 so the user stops using it
 ▼
 and the recall problem remains

BREAKING IT: move the judgement from the human to the machine.
 That is a much harder engineering problem, and it is the
 problem this project actually solves.

Why manual curation defeats the purpose. This realisation is the single most important conceptual step in the project, and everything downstream follows from it.

The reframed goal therefore became: *capture everything automatically, decide relevance algorithmically, and answer questions in natural language afterwards*. The user should never perform a filing action. This moves the entire difficulty of the project into one place — the quality of the machine's judgement about what matters — and that is where the research contribution later emerged.

3. Personal knowledge management: the theory

Personal knowledge management (PKM) is a body of practice, developed largely outside academic computer science, concerned with how an individual captures, organises, retrieves and expresses what they know. Three frameworks from that tradition shaped this system's architecture, and each maps onto a concrete implementation decision.

PARA — Projects, Areas, Resources, Archives

Devised by Tiago Forte, PARA argues that information should be filed by *actionability* rather than by subject. A Project is something with a deadline and an outcome. An Area is an ongoing responsibility with no end date. A Resource is material of future interest. An Archive is anything inactive. The insight is that subject-based filing decays, because the same subject appears in multiple contexts, whereas actionability-based filing stays stable.

The machine analogue implemented here is the **importance score**, discussed in detail in Volume IV. A memory's score includes a *future utility* term that rewards actionable content (deadlines, instructions, invoices) and project-linked content, and the *temporal* term decays with age so that inactive material sinks naturally. The system does not maintain folders, but it reproduces the essential distinction PARA makes: active material ranks above archival material, without the user deciding which is which.

CODE — Capture, Organise, Distil, Express

Forte's second framework describes a four-stage pipeline through which information passes. This maps onto the system's seven-stage pipeline almost directly, with the added stages being machine-specific:

CODE stage	Human activity	Implemented as
Capture	Record the thing before it is lost	Screenshot capture with OCR; IMAP mail ingestion; iCalendar ingestion; web and social scrapers
Organise	Put it somewhere it can be found again	Curation scoring, memory cards, the knowledge graph, and the retrieval index

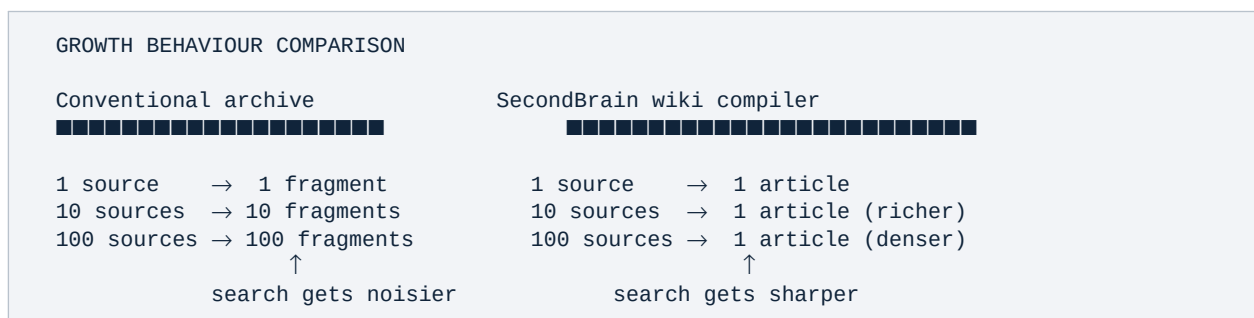
CODE stage	Human activity	Implemented as
Distil	Reduce it to what actually matters	The wiki compiler, which maintains one article per topic domain and rewrites it as new evidence arrives
Express	Produce something new from it	Question answering over accumulated memory; generated deliverables; spoken briefings

Table 2 — The CODE pipeline and its implementation. The two additional pipeline stages not present in CODE — Retrieve and Learn — exist because a machine system must search its own store and update itself.

Progressive summarisation

The practice of repeatedly condensing a note — highlight the important sentences, then bold the important words within those, then extract the essence into a summary. The theory is that a note becomes more valuable as it becomes shorter, because its information density rises and it can be re-read quickly.

This is implemented as the **wiki compiler**, and it is worth examining because it produces a counter-intuitive property. As new material is ingested, the compiler does not append. It *rewrites* the topic article, folding the new evidence into the existing synthesis. The corpus therefore becomes shorter and better as it grows, which is the opposite of how a conventional archive behaves, and it is what makes question answering over long-term memory tractable at all — a bounded set of dense articles is searchable; an unbounded set of fragments is not.



Progressive summarisation applied at corpus scale. The measured result at the time of writing was 93 compiled articles from several hundred ingested items.

What PKM theory supplied, and what it could not

The PKM tradition supplied the *stages* and the intuition that summarisation adds value. It could not supply a formal criterion for importance, a mechanism for handling contradiction, or a way to measure whether any of it worked. Those gaps are precisely where the project became engineering and research rather than practice, and they are addressed in Volumes IV and VII.

4. The reference point, analysed properly

The author's stated design target was JARVIS, the assistant from the Iron Man films, and two reference videos were supplied: the workshop scene from *Iron Man 2*, and a compilation of the abilities of Tony Stark's several artificial intelligences. The author's instruction was to watch them and report what was understood.

It is worth recording that the videos could not be watched directly — neither carries subtitles, so transcript extraction returned nothing, and no video playback was available. The analysis was therefore performed from the titles, descriptions, and a first-principles examination of what the depicted assistant actually does. Rather than obscure this, the report states it, and the analysis below stands on its own reasoning.

The superficial reading of those films is that JARVIS is a voice assistant with holographic displays. Acting on that reading produces a system that is a talking interface with animations — visually impressive, functionally empty, and a very common failure mode in student projects that cite this reference. The useful reading examines what the character *does* and identifies five behaviours:

Behaviour	What it looks like on screen	Why it matters
There is no interface	Stark walks in and speaks. Nothing is launched, nothing is opened.	The assistant is ambient rather than an application. Any design requiring the user to navigate to it has already failed this test.
It speaks first	JARVIS initiates — reporting status, raising problems, announcing arrivals.	A conventional assistant only ever responds. Initiative is the property that distinguishes an assistant from a search box.
It holds state	It knows what Stark was doing before he entered the room.	Requires persistent memory across sessions, which is the entire premise of a second brain.
It acts	It does not report that the simulation is missing; it runs the simulation.	Turns the system from an advisor into an instrument. Requires write access, which carries risk and is therefore deliberately constrained here.
It has judgement	It warns, objects, and occasionally refuses.	Requires the system to hold opinions about its own knowledge — which is what confidence scoring, contradiction detection and gap detection provide.

Table 3 — The five behaviours, separated from their visual presentation. The analysis concluded that the voice is the least essential element.

The conclusion drawn from that analysis shaped the later build phases decisively. If the voice were the essence, effort should go into speech quality and visual effects. If *presence* and *initiative* are the essence, effort should go into being addressable by name and speaking without being asked. The second reading was adopted, and it is why the project later implemented offline wake-word detection and proactive spoken announcements rather than further animation work.

5. The author's brief: every constraint recorded

Constraints are recorded here in the order they were given, because each eliminated design options and the elimination is what produced the final architecture. This table is the closest thing in this report to a requirements specification.

#	Constraint as given	Consequence for the design
1	Use a free local language model; do not pay for OpenAI	All inference runs through Ollama on the laptop's GPU. Removes per-token cost and keeps data on-device, but caps model size to what fits in 6 GB of VRAM, which in turn motivated the decision to make every model-dependent feature optional.
2	It must not look like a project interface; it must feel like a personal AI	Rejected dashboards, side banners and admin-panel aesthetics. Directly caused three successive interface rebuilds (deviation D5).
3	Every module must be visible as a real tab; do not hide functionality	Produced the fifteen-tab operating-system shell, and eliminated an earlier design that used a single dashboard with drawers.
4	No fake functionality — if a service is not connected, say so	Removed all seeded demo data from interactive screens. This single instruction is why the Gmail and Calendar screens show explicit disconnection states and why the demo rows in the task list were later removed. It also shaped the research layer: measured results only.
5	Build the scraping software yourself; do not pay for Apify or SupaData	Produced a native ingestion stack on free libraries, and established the principle applied later to mail and calendar (deviation D7) that saved a recurring verification cost.
6	The assistant must be reachable by name, with a transparent floating structure	Produced the transparent always-on-top orb window and later the offline wake-word implementation. 'Transparent' required changes at three layers simultaneously.
7	Do not use Marvel-copy material; it should feel futuristic but be original	The assistant is named and addressed distinctively, and the visual language is derived from generic science-fiction conventions rather than film assets.
8	The project must satisfy an IEEE paper requirement	Forced the addition of a research layer: baseline comparison, quantitative measurement, and honest limitation reporting. This constraint is what turned a good application into a defensible contribution.

#	Constraint as given	Consequence for the design
9	I need the paper to be genuinely novel, and the project must actually be what the paper claims	Produced the pre-publication audit which found six of seven claimed contributions unimplemented, and caused the implementation-first ordering recorded as deviation D9.

Table 4 — The author's constraints and their direct architectural consequences. Constraint 9 is the one that most changed the project.

On the relationship between author and implementation

Nine of the thirteen documented deviations were driven by a constraint in this table. The author's role was consistently to supply the *requirement and the judgement* — what the system must do, what would be unacceptable, and when a result was not convincing — while the implementation explored the technical space and reported honestly when it found that a stated plan was wrong. The most productive interactions were the two questions that challenged an assumption directly: ‘*why can Grok connect and this cannot?*’, which exposed a US\$540–\$1,800 recurring cost and led to a completely different integration strategy; and ‘*why does it answer once and then stop?*’, which exposed a missing event handler that had been invisible in review.

Volume II — Architecture

6. Design philosophy

Four principles governed every architectural decision. They are stated first because the specific choices in the following sections are all consequences of them.

Principle	Statement	Consequence if violated
Optionality	No single dependency is load-bearing. Every model, index, integration and library is imported conditionally and degrades to a working fallback.	A single missing package makes the application unusable. This was observed early: the system would not start because one undeclared package was absent.
Additivity	New capability is added alongside existing capability, never by rewriting it. The research layer observes memories through foreign keys and modifies nothing in the capture pipeline.	The working system breaks while being extended, which is the most expensive kind of failure because the working state is lost.
Testability by construction	Every component must be exercisable without the components it depends on. Storage, retrieval, extraction and policy are separated so each can be driven directly.	Components can only be tested end-to-end, so a failure cannot be localised.
Honest degradation	When something is unavailable, the system says so and continues. It never substitutes plausible-looking content for a failed operation.	The user cannot distinguish working from broken, which is the defect class that consumed the most time in this project.

Table 5 — The four architectural principles.

7. The seven-stage pipeline



The pipeline. Stage ⑦ ('LEARN') feeds back into stages ② and ③, which is what makes the memory adaptive rather than a static archive.

Stage	Responsibility	Implemented in	Failure behaviour if unavailable
① Capture	Acquire raw material: screen frames, email, calendar events, web pages, social posts, voice input	capture/, integrations/, services/	System runs; other sources continue; the unavailable source reports its state
② Understand	Extract meaning: OCR text, entities, topics, temporal facts, importance scores, contradictions	agents/, extraction/, entities/, research/	Falls back to keyword extraction; temporal and conflict features disable themselves
③ Store	Persist to two independent stores: relational (49 tables) and the Git-backed vault (JSON cards, images, wiki, graph)	models/, memory/, agents/vault_agent.py	Vault sync is retried in the background; local persistence is unaffected
④ Retrieve	Select relevant context. Four interchangeable strategies: dense, hybrid, graph, adaptive	search/, semantic_search/, ranking/, research/pipeline.py	Falls back through dense → TF-IDF → keyword; never returns nothing silently
⑤ Reason	Generate an answer from the assembled context	rag/, llm/, prompts/	Falls back to extractive summary of the retrieved context, and says which mode it used

Stage	Responsibility	Implemented in	Failure behaviour if unavailable
⑥ Explain	Deliver the answer: text, speech, notification, task	routes/, websocket/, voice/, proactive	Each delivery channel fails independently
⑧ Learn	Update importance, detect gaps, consolidate duplicates, close superseded facts	research/	Feature disables itself; retrieval continues with the previous state

Table 6 — The seven stages, with the modules that implement them and the degradation path if each is unavailable. The fourth column is the practical expression of the optionality principle.

8. The data model in full

The database contains 49 tables. They divide into five groups, and the grouping mirrors the chronology of the build: capture came first, then knowledge organisation, then conversation, then platform, and finally the research layer.

CAPTURE · 11 tables

memory_sessions	a capture period, typed by activity
screenshots	raw frames, pruned after the hero is elected
activity_logs	window and application focus events
app_usage	per-application time within a session
clipboard_logs	copied text as a knowledge signal
extracted_text	OCR output per frame
ocr_metadata	OCR engine state and confidence
processed_sessions	sessions whose OCR pass is complete
detected_topics	topics identified within a session
semantic_chunks	the unit of embedding and retrieval
settings	per-user configuration

KNOWLEDGE · 17 tables

memories	the central record: title, content, hash, source, topic
memory_tags	tags attached to a memory
memory_relationships	typed links between memories
search_index	the full-text surface used by keyword retrieval
session_summaries	a summary per capture session
graph_nodes	entities in the knowledge graph
graph_edges	typed relationships between entities
concept_clusters	grouped nodes
graph_metadata	build state of the graph
topic_relationships	topic-level links
learning_progression	ordered topic sequences
vector_memories	the embedding record per memory
memory_clusters	semantic clusters over memories
semantic_relationships	similarity-derived links
embedding_jobs	the embedding work queue
search_history	every query, for evaluation and personalisation
timeline_events	chronological event log

CONVERSATION · 11 tables

conversations	a question-answering thread
messages	individual turns
retrieved_memories	which memories were used for which answer (provenance)
conversation_context	assembled context per turn
ai_summaries	generated summaries
ai_feedback	user ratings, the personalisation signal
voice_sessions	spoken interaction sessions
transcripts	speech-to-text output with confidence
conversation_audio	generated speech artefacts
language_preferences	language, voice and wake-word configuration
voice_commands	recognised commands and their outcomes

RESEARCH · 7 tables ← added last; observes memories, never owns them

memory_scores	the eight-term importance score per memory
temporal_facts	(subject, predicate, object) with validity intervals
memory_conflicts	detected contradictions and their resolution policy
memory_consolidations	clusters merged, with member identifiers retained
forgotten_memories	tombstones for unlearned content
knowledge_gaps	detected exposure-without-depth concepts
research_runs	benchmark executions and their metrics

PLATFORM · 3 tables

users	accounts
sessions	authentication sessions
activities	the audit log

The complete schema. The research group is shown last because it was built last, and because its position in the design is deliberately peripheral: it observes the knowledge group and is removable without affecting it.

The dual-store decision

Every memory is written to two independent stores. The relational store is fast, queryable and holds the derived research state. The Git-backed vault holds immutable JSON cards, compressed hero images, compiled wiki articles and the knowledge graph, and is synchronised to a remote repository every sixty seconds. The duplication is deliberate and the reasoning is durability: if the database is corrupted, the vault reconstructs it; if the vault is lost, the database survives. Neither is a backup of the other — they are genuinely independent representations with different shapes, and the system is designed so that either can be rebuilt from the other.

9. The agent layer

Intelligence is distributed across ten specialised agents rather than concentrated in one monolithic service. Each agent has a single responsibility, its own failure mode, and no knowledge of the others' internals. The reasoning is the same as for the pipeline stages: a large component that does everything cannot be tested, instrumented or replaced.

Agent	Trigger	Responsibility	If it fails
Curation	Every capture	Score captured text for information density; reject low-value frames before they are stored	Every frame is stored; storage grows and retrieval noise increases
Vault	On card creation	Write JSON cards, compress hero images, commit and push to the remote repository	Memory exists only locally; a background retry loop attempts sync
Wiki compiler	On card creation	Fold new evidence into the per-topic article, rewriting rather than appending	Knowledge accumulates as fragments and is never distilled
Insight	Periodic sweep	Detect collisions between separate topics; raise proactive alerts	The system answers but never volunteers anything
Social ingestion	On demand or scheduled	Extract transcripts and articles from video, social and web sources	Those sources must be read manually
Mail ingestion	Scheduled interval	Read new mail read-only; create memories, tasks, notifications and spoken alerts	The inbox remains separate from the knowledge base
Deliverable	On demand	Generate documents synthesised from accumulated memory	The system can explain but cannot produce artefacts
Proactive voice	On agent event	Apply priority, quiet-hours, cooldown and dedupe policy, then speak	Alerts are filed silently and the assistant loses initiative
Wake word	Always listening	Score audio frames; fire a wake event on the target phrase	The hotkey still summons the assistant; hands-free is lost
Embedding worker	On memory creation	Queue and compute embeddings in the background	Semantic retrieval falls back to TF-IDF; the system continues

Table 7 — The ten agents. Note that every entry in the final column is a degradation rather than a failure: this is the optionality principle applied at the agent level.

10. The API surface

The backend exposes 162 endpoints across 150 paths, grouped into 19 route modules. The surface is larger than a typical application of this size because it is the boundary at which every capability is testable — and the test strategy depends on being able to drive each capability independently.

Group	Purpose	Representative endpoints
health, auth, sessions	Liveness, login, session lifecycle	GET /api/health · POST /api/auth/login · POST /api/sessions

Group	Purpose	Representative endpoints
capture	Screen capture control and retrieval	POST /api/capture/start · GET /api/capture/status · GET /api/capture/live-preview
ocr	Text extraction pipeline	GET /api/ocr/texts · POST /api/ocr/sessions/{id}/process · GET /api/ocr/topics
memory	The archive: sessions, memories, relationships, timeline	GET /api/memory/search · GET /api/memory/timeline · POST /api/memory/rebuild
semantic	Embedding search and clustering	POST /api/semantic/hybrid-search · GET /api/semantic/status · GET /api/semantic/clusters
graph	Knowledge graph and vault	GET /api/graph/nodes · GET /api/graph/vault/wiki · POST /api/graph/vault/sync
chat	Question answering with provenance	POST /api/chat/ask · GET /api/chat/conversations/{id}/retrieved
voice	Speech sessions, transcripts, preferences	POST /api/voice/sessions · GET /api/voice/status · PUT /api/voice/preferences
activities, timeline, settings	Audit log and configuration	GET /api/activities · GET /api/timeline · PUT /api/settings
social	Native content ingestion	POST /api/social/ingest-url · GET /api/social/status
os	Tasks, reminders, calendar, projects, automations, activities, notifications	GET/POST /api/os/tasks · GET /api/os/intelligence · GET /api/os/notifications
google	OAuth-connected Gmail and Calendar	POST /api/google/connect · GET /api/google/callback · GET /api/google/gmail/messages
mail	Unified mail and calendar, provider-agnostic	POST /api/mail/imap/connect · GET /api/mail/messages · POST /api/mail/sync
proactive	Wake word and proactive voice control	GET /api/proactive/status · POST /api/proactive/wake/test · POST /api/proactive/announce
research	The seven research contributions	POST /api/research/benchmark · POST /api/research/forget · GET /api/research/temporal
WebSocket	Live push: capture events, agent activity, wake, speech	/ws/live · /ws/voice · /ws/chat

Table 8 — The API surface by group. The 'mail' and 'google' groups are deliberately parallel: the first abstracts over both connection methods, the second exposes the OAuth path directly for accounts where it is the only option.

11. The desktop shell and interface

The application is an Electron desktop shell containing a React interface. Electron was not a preference but a requirement: the floating assistant must be a borderless, transparent, always-on-top window positioned over the desktop, and a browser cannot create one. Given that constraint, the interface is ordinary React with hash routing.

Surface	Implementation	Notes
Main window	Framed Electron window, hash-routed React application	Nineteen routes. Fifteen are primary navigation tabs; the remainder are aliases and the standalone orb.
Assistant orb	Second BrowserWindow: frameless, transparent, always-on-top, not in the taskbar, bottom-right	Transparency required simultaneous changes to Electron's window flags, the document root CSS and the body background (see defect D16).
Tray icon	Persistent with a visibility toggle	Added after the application became invisible when hidden with no way to restore it.
Command palette	Ctrl+K overlay with backend search	Provides keyboard access to every destination without navigating the rail.

Surface	Implementation	Notes
Live channel	WebSocket from backend to renderer	Carries capture events, agent activity, wake events and speech requests. The renderer holds no policy — it renders what the backend decides.

Table 9 — The interface surfaces. The division of responsibility matters: the backend decides what should happen, the renderer decides how it looks, and neither duplicates the other's logic.

12. Technology selection and its trade-offs

Each choice below is recorded with the alternative that was rejected, because the reason for a decision is only visible against what it displaced.

Decision	Chosen	Rejected alternative	Reason
Language model	Ollama with a small local model	Hosted API (paid)	Author constraint 1. Local inference also keeps all personal data on-device, which matters because the corpus is a life record.
Model transport	Ollama's native API	Its OpenAI-compatible endpoint	The compatibility layer returned 'model not found' for models that existed. A shim adds a failure mode without adding capability.
Primary store	SQLite	PostgreSQL or MySQL	Single-user, single-machine. A server database would add deployment complexity for no benefit at this scale.
Vector store	ChromaDB, optional	FAISS, Qdrant, Pinecone	Local and persistent by default. Made optional so the system runs without it.
Dense retrieval fallback	TF-IDF cosine	Require an embedding model	Determinism and zero-download startup. It also made the benchmark reproducible, which the research required.
Mail access	IMAP with app passwords, plus OAuth as a secondary path	Google OAuth only	The default path must not require a Google Cloud project, a consent screen, or an annual security assessment. See deviation D7.
Calendar access	Private iCalendar address	Google Calendar API	Requires no authentication at all, and is read-only by construction.
Wake word	openWakeWord, offline	Cloud keyword spotting (Porcupine, Google)	No audio leaves the machine, no account, no per-request cost. The trade-off is that it is a keyword spotter, not speech recognition.
Speech recognition	Browser Web Speech API	Whisper on-device	Zero setup and no model download. Whisper remains available through the optional pack for offline accuracy.
Interface framework	React 18 with hash routing	Server-rendered or vanilla	Electron requires a static bundle; hash routing avoids file-protocol path issues.
Desktop shell	Electron	Tauri or a native application	Required for a transparent overlay window. Tauri was considered but the transparency requirements were better supported by Electron at the time.

Table 10 — Every significant technology decision, with the rejected alternative and the reason. This table is the clearest single statement of the project's engineering philosophy.

The pattern across every decision

In almost every case the rejected alternative was the *more capable* option, and it was rejected for a reason of ownership, cost, or failure visibility. A hosted language model is more capable than a 3B local model. Google's Calendar API is more capable than an iCalendar feed. Cloud speech recognition is more accurate than the browser's. In each case the less capable option was chosen because it could be owned outright, could not generate a recurring bill, and could not fail in a way the user would not understand. For a personal system holding a life record, those properties outweighed capability.

Volume III — Build Chronology

Eight phases. For each: what the author asked for, what was built, the reasoning behind the approach, and — deliberately — what went wrong. The failures are included because the defects of each phase reveal what was not yet understood, and several of them directly motivated the phase that followed.

Phase 0 — Foundations

August 2026

Request. Build a personal knowledge system that captures what I do on my laptop and lets me search it later.

What was built.

- FastAPI backend with SQLAlchemy and Pydantic schemas.
- SQLite schema with users, sessions, activities and capture tables.
- JWT authentication with password hashing.
- Screenshot capture scaffolding and text-to-speech plumbing.
- WebSocket scaffolding for live updates.

Reasoning. The decision that mattered was to build capture as a first-class pipeline rather than as a feature of a notes application. A notes application begins with an empty page and waits; a capture pipeline begins with a screen and a clock. Everything downstream — curation, scoring, retrieval — only makes sense when there is a steady supply of raw material.

What went wrong.

- The virtual environment was not located reliably when the backend was launched from Node. The backend appeared to start but had no dependencies installed. This produced a confusing failure where the server was running but every import failed.
- Exceptions were swallowed in several endpoints, so operations that had failed returned empty results rather than errors. This is the first appearance of the defect class that would eventually consume more time than any other.

Phase 1 — Capture, Curation and the Vault

17 September

Request. Make the captured knowledge persistent, visual and safe from loss; put it in GitHub; and make the stored material actually meaningful rather than a dump of screenshots.

What was built.

- An agentic curation layer scoring captured text for information density.
- A best-shot hero image engine electing one representative frame per information period, compressed to WebP.
- Ephemeral screen pruning: raw frames deleted after the hero is extracted, so persistent storage for visual data approaches zero.
- A Git vault agent committing memory cards and images automatically.
- A self-improving wiki compiler maintaining one article per topic domain.
- A three-dimensional neural visualisation on canvas.
- Native Ollama streaming, replacing a compatibility endpoint that was returning 404s.

Reasoning. Three decisions here proved consequential. The hero image approach inverts the storage problem: rather than keeping all evidence, keep the best evidence, and make 'best' a measurable criterion

(text density). The wiki compiler produces the counter-intuitive property that the corpus becomes shorter and better as it grows. And routing the model through Ollama's native API rather than its compatibility shim established a principle applied repeatedly afterwards: when talking to your own infrastructure, avoid adapters.

What went wrong.

- The vault agent resolved the repository root incorrectly, committing to the wrong directory — a path-resolution defect that recurred in other components.
- Titles were lost for web and video captures because title extraction depended on OCR text that is sparse for those source types. Fixed by falling back to page metadata.

Phase 2 — The Intelligence Layer

18 September

Request. Stop using paid scraping services — build that capability yourself. Also, the system should notice things and tell me, rather than waiting to be asked.

What was built.

- A native ingestion stack replacing the paid services: video transcripts via a free library, social posts via a public endpoint, image-sharing sites via an open-source client with a metadata fallback, and general web articles via HTML parsing.
- A proactive insight layer detecting collisions between otherwise unrelated topics.
- A morning briefing compiled from accumulated state and spoken aloud.
- A deliverable generator producing documents from memory.
- An ambient command overlay bound to a global shortcut.

Reasoning. The author's instruction contained a generalisable principle: replace a recurring cost with a one-off engineering effort. The paid services were the obvious path and the author rejected them; the replacement was built. The same reasoning was later applied without prompting to mail and calendar, where it avoided a recurring verification cost entirely (deviation D7). Performance work here also demonstrated that the bottleneck was architectural rather than computational: parallelising independent network fetches took ingestion from several seconds to 0.98 seconds without changing any algorithm.

What went wrong.

- Repeated hero-image extraction meant raw frames could be deleted while a live preview still referenced them, producing broken images. Fixed with a dedicated preview buffer excluded from pruning. The general principle: never let a cleanup process delete the last reference to something a view depends on.
- Window-title tracking was eagerly initialised at import time, which slowed startup and caused failures on machines without the necessary platform bindings. Made lazy.

Phase 3 — The Interface

21 September

Request. This does not feel like a personal AI. It feels like a project dashboard. Make it feel like the reference, and do not hide any of the functionality.

What was built.

- Three successive interface rebuilds in one day.
- The first transformed the shell into a graph-oriented workspace matching a reference screenshot the author supplied.

- The second removed the project sidebars entirely and introduced a living holographic core that tracks the mouse, with an ambient voice intercom.
- The third produced the definitive fifteen-tab operating-system shell with a universal command terminal.
- A transparent, borderless, always-on-top orb window positioned at the bottom-right of the desktop.
- Procedural audio for interface feedback.

Reasoning. The iterative rebuilds were convergence, not indecision. The requirement could not be specified in advance because the author could not articulate the difference between a dashboard and an assistant without seeing the alternatives. The first attempt established that the visual language was wrong; the second established that removing navigation entirely makes functionality unfindable; the third reconciled both. This is a general pattern in interface work: the requirement is discovered by building the wrong thing quickly.

What went wrong.

- A missing component import produced a blank screen on the home route. An error boundary was added in response, which displayed the exact cause and became a permanent safety net.
- Making a window genuinely transparent on Windows required simultaneous changes at three layers — the Electron window flags, the document-root CSS, and the body background. Changing any one alone produced the dark rectangle the author reported.
- The auto-launch registration pointed at the bare runtime executable in development mode, so a Windows boot produced a raw framework splash screen rather than the application.

Phase 4 — Stabilisation

22 September, morning

Request. “It’s an error. I need you to fully do an error check environment and fix every error.”

What was built.

- No features. Instrumentation only.
- The first audit: a linter configured to fail on undefined identifiers, plus eight static integration checks covering imports, component existence, client method definitions, route-module existence, frontend-to-backend URL agreement, preload surface completeness, Python compilation, and route file resolution.
- The second audit: a live endpoint sweep that boots the server, logs in, and calls every registered route, reporting status and timing.
- A route-render smoke test that renders and mounts every interface route.
- An API contract checker comparing every URL the interface calls against every route the backend serves.

Reasoning. The difference between the two audits is the most important methodological lesson in this project. The first audit was correct and useful and could not possibly have found the most serious defect, because a login function that raises before comparing a password is syntactically valid code. Only running the system and attempting to log in revealed it. This is why the verification system described in Volume VIII ends with a live sweep rather than a linter.

What went wrong.

- **Authentication had never worked.** A service referenced a configuration object without importing it, so the login function raised before comparing any password. The frontend swallowed the resulting error, so the application ran with no token and 52 protected endpoints returned 401 — which the interface rendered as empty states. Everything looked fine and nothing worked.
- Thirty-three call sites used a raw browser fetch which does not attach the authentication token, so even a correct login would not have authenticated most screens.

- A test was passing while the system it tested was offline, because it checked only the status code of an endpoint that always returns success.

Phase 5 — Connectivity

22 September

Request. Connect these three email accounts to my Second Brain. And then, separately: why can a commercial assistant connect directly while this cannot?

What was built.

- Google OAuth with multi-account support, encrypted token storage and strictly read-only scopes.
- Then, after research into what publishing such an application requires: a second connection path using IMAP with app passwords for mail and the private iCalendar address for calendars.
- A unified mail and calendar API abstracting over both providers, so the interface never needs to know how an account was connected.
- Encrypted credential storage that refuses to write inside the synchronised vault directory.
- An ingestion agent reading new mail read-only and turning each message into a memory, a task, a notification and — where a deadline is detected — a spoken alert.

Reasoning. This phase contains the project's most consequential decision, and it came directly from the author's question. Investigating why a commercial assistant appears to connect 'directly' revealed that it does not: it performs the same consent flow, and appears effortless only because the vendor has absorbed the cost of a mandatory third-party security assessment that runs to several hundred dollars annually with re-certification every year. For an application with one user that cost is indefensible, and staying unverified is worse in practice because refresh tokens expire weekly. The resolution was to recognise that OAuth is not the only way to read a mailbox: IMAP with an app password is what desktop mail clients have used for decades, is free, does not expire, and is read-only by construction when the correct commands are used.

What went wrong.

- The author's institutional account cannot use app passwords, because the provider disabled them for managed domains in May 2025 and the institution may block third-party access regardless. Documented honestly rather than worked around; the system was designed to function with any subset of accounts connected.
- The ingestion test initially registered the author's real address, which later caused a serious incident described in Volume VI.
- An error-mapping defect caused six endpoints to report an upstream failure when the real problem was a non-existent identifier.

Phase 6 — Presence

22 September

Request. The floating orb answers one question and then stops. Also: here are two reference videos — report what you understand from them.

What was built.

- An analysis of the two videos separating their surface (voice, holograms) from their substance (presence, initiative, state, action, judgement).
- An offline wake-word implementation so the assistant can be addressed by name.
- Proactive speech with a policy layer: priority threshold, quiet hours, per-source cooldown, and sentence deduplication.

- A repair of the orb's voice loop, which had no handler to restart recognition after the browser ended the session.

Reasoning. The orb defect was diagnosed as a missing re-arm: the browser's speech recogniser ends its session by itself, and the code had no handler to restart it, so the microphone came up exactly once. Critically, the summon gesture only spoke a greeting and never restarted the recogniser, so summoning the assistant repeatedly produced repeated greetings from a dead microphone. The wake word required validating a real audio model rather than a mock: synthetic speech was generated, decoded to the correct sample rate, and pushed through the project's own scoring function, producing a measured separation between the positive case and every negative control.

What went wrong.

- Three further ways the assistant could become unresponsive, two of which had never been observed and were found by writing the tests: a turn the model never answers leaves the microphone paused indefinitely; the text-to-speech completion event is not guaranteed in Electron when the window is hidden; and the existing wake handler toggled window visibility rather than showing it, so waking an already-visible orb would hide it.
- The wake-word library's loading interface differed between the installed version and the documented version, so the first implementation would have failed silently on the target machine.

Phase 7 — The Research Layer

22 September

Request. I need a paper that is genuinely novel, and the project must actually be what the paper claims.

What was built.

- An audit of the paper's claims against the codebase before any paper text was written.
- Seven components implementing the claimed contributions: adaptive memory scoring, a temporal knowledge graph, contradiction-aware retrieval, memory consolidation, knowledge-gap detection, retrieval-enforced forgetting, and a benchmark.
- A synthetic corpus and question set covering six categories.
- A four-system comparative evaluation with custom metrics for staleness, leakage and duplication.
- An IEEE-format paper generated from measured output rather than hand-written numbers.

Reasoning. Before writing any paper text, the seven claimed contributions were audited against the codebase. One partially existed; six did not exist at all. Writing the paper first would have produced a document describing software that had not been written. Building the components then produced the project's most interesting finding, which was not the one anticipated: the research layer did not improve retrieval accuracy, because accuracy was already saturated at 1.000. What it improved was the ordering of results and the staleness of the top-ranked item — and that is a far more interesting result, because it means the conventional metric would have reported that seven components accomplished nothing.

What went wrong.

- Nine defects found by the research tests, including a temporal modelling error that caused the exact failure the paper is about, and a substring-matching defect that flagged any note mentioning 'notes' as a logical contradiction.
- The benchmark was not reproducible between runs because derived state from a previous run perturbed the following one.
- The mail-ingestion test was discovered to have deleted 123 of the author's genuine memory cards. Caught before commit, restored from version control, and the test made structurally incapable of touching unmarked data.

The chronology in one observation

Phases 4 to 7 all occurred on the same day. The pace was not the result of hurrying; it was the result of the verification system built in Phase 4. Before it existed, changes were risky and had to be validated by hand. After it existed, a change could be made, checked against fifteen gates in under a minute, and committed with confidence. Tooling compounds: the afternoon of an engineering project is faster than the morning only if the morning was spent building the means to verify.

Volume IV — Concepts and Theory

Fourteen chapters covering the theory the system rests on. Each is written for a reader who does not already know the terminology: the concept is introduced from first principles, its mathematics is given where it is short enough to be useful, and it is then connected to the specific place in this codebase where it is applied.

13. Information retrieval: from keywords to meaning

Information retrieval is the discipline of finding documents relevant to a query. It is older than machine learning, and its classical results remain the foundation of modern systems.

The lexical tradition.

Early retrieval systems matched query terms against document terms. The naive version — count how often each query word appears — fails badly, because common words appear everywhere and carry no information. The classical remedy is **inverse document frequency** (IDF), which weights a term by how rare it is across the corpus:

$$\text{idf}(t) = \log((1 + N) / (1 + \text{df}(t))) + 1$$

where N = number of documents

df = number of documents containing the term

Weight each term: $w(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$

Score a document: cosine similarity between query and document weight vectors

TF-IDF. A term appearing in every document contributes almost nothing; a term appearing in three documents is highly informative. This is implemented in SecondBrain as the dense-retrieval fallback when no embedding model is present.

The weakness of the lexical approach is **vocabulary mismatch**: a query for “vehicle” will not retrieve a document about “cars”, because the two share no terms, even though a human considers them the same question. This is the problem that vector representations solve.

The probabilistic tradition.

A second classical line treats retrieval as probability estimation: rank documents by the probability they are relevant to the query. The practical outcome of that line is BM25, which adds term-frequency saturation (the tenth occurrence of a word adds less than the second) and document-length normalisation to the TF-IDF weighting. BM25 remained a strong baseline for decades and is still competitive with neural methods on many benchmarks.

The neural tradition.

Neural retrieval replaces term weighting with learned representations. Instead of counting words, a network maps both query and document into a vector space in which proximity means similarity of meaning. Because the mapping is learned from data rather than derived from term statistics, it handles vocabulary mismatch: “vehicle” and “cars” land near each other because they appear in similar contexts during training.

Hybrid retrieval.

Lexical and neural retrieval fail in different circumstances. Neural retrieval misses exact matches — a query for an error code or a proper noun may not benefit from semantic generalisation. Lexical retrieval misses paraphrase. Hybrid retrieval fuses both rankings, typically by weighted score combination or rank fusion. SecondBrain’s hybrid retriever combines a dense candidate list with a keyword candidate list and re-ranks, which is the second of the four pipelines evaluated in Volume VII.

Approach	Strengths	Weaknesses	Where it is used here
Boolean / exact	Precise, predictable, no model needed	No ranking, no partial match	Not used directly

Approach	Strengths	Weaknesses	Where it is used here
TF-IDF / BM25	Fast, deterministic, no training, interpretable	Vocabulary mismatch, no synonymy	The dense-retrieval fallback; also the deterministic basis of the reproducible benchmark
Dense neural	Handles paraphrase and synonymy	Needs a model, misses exact tokens, can be non-deterministic across versions	The primary dense path when an embedding model is configured
Hybrid	Best of both; robust to query type	Two systems to tune and maintain	The baseline against which the proposed pipeline is measured

Table 11 — The retrieval traditions. The project spans all four for a specific reason: the research evaluation requires a fair baseline, and a baseline that degenerates to zero when a model is absent would flatter the proposed method.

14. Embeddings and the geometry of meaning

An **embedding** is a function mapping text to a fixed-length vector of real numbers, trained so that texts with related meanings map to nearby points. The space typically has between 128 and 1536 dimensions.

```
"I want to learn Java"      → [ 0.21, -0.83,  0.44, ... ] ■
"What language am I studying?" → [ 0.19, -0.79,  0.41, ... ] ■ close
"Which language am I learning?" → [ 0.22, -0.81,  0.46, ... ] ■
"The cat sat on the mat"     → [-0.55,  0.12, -0.90, ... ] far
```

Similarity: $\cos(\theta) = (A \cdot B) / (|A| \cdot |B|)$ in $[-1, 1]$

The cosine is used rather than Euclidean distance because it measures direction only, ignoring magnitude — so a short and a long passage about the same subject still compare as similar.

Embeddings convert semantic similarity into geometry. This is the enabling abstraction behind retrieval by meaning.

Three properties matter in practice. **Dimensionality** trades representational capacity against memory and search cost. **Normalisation** — dividing each vector by its length to unit magnitude — makes the cosine equal to the dot product, which is computationally cheaper and is why the sentence-transformer models used here are configured with normalisation enabled. **Anisotropy** is the tendency of learned embeddings to occupy a narrow cone rather than filling the space; this depresses absolute similarity values and is why absolute thresholds must be calibrated per model rather than copied between systems — a fact that directly caused defect D22 in this project.

Why absolute similarity thresholds are dangerous

The consolidation component must decide when two memories are the same. The threshold commonly recommended in practice is 0.90 cosine similarity. Measuring the actual distribution in this corpus showed that genuinely reworded duplicates — the common real case — score around 0.82, while unrelated memories score below 0.30. A 0.90 threshold missed every reworded duplicate. The lesson is general: *a similarity threshold is a property of a particular embedding space, not a universal constant*. It must be calibrated from the measured distribution of the data it will be applied to.

15. Chunking, context windows and the retrieval budget

A language model can only attend to a bounded amount of text at once — its **context window**. A personal corpus is far larger than any context window, so retrieval must select what to include. The amount of text a retriever supplies is therefore a **budget**, and every retrieved item consumes part of it.

This produces three consequences that shape the design. First, **chunk size** is a trade-off: small chunks embed more precisely but fragment meaning and require more of them; large chunks preserve context but dilute the embedding, since one vector must represent everything in the passage. Second, **duplicate content is not free** — three copies of the same paragraph consume three times the budget and contribute one unit of information. This is the argument for consolidation, and it is measured in this project as the

duplicate rate. Third, **irrelevant but plausible** content is actively harmful, because it consumes budget and can mislead the generation step, which is the argument for the staleness and contradiction penalties.

THE RETRIEVAL BUDGET

context window 
 spent on  instructions retrieved passages

In a naive retriever the passages region fills with:

- three copies of the same page → duplicate rate
- a fact that was true eight months ago → stale@1
- content the user asked to delete → leak rate

Each is measurable, and each is what the three custom metrics report.

Why duplication, staleness and leakage are not merely inefficiencies but budget theft. This framing is what motivated the custom metrics.

16. Knowledge representation and graphs

A vector store answers “what resembles this?” A knowledge graph answers “how are things related?” These are different questions and neither store subsumes the other.

A **graph** consists of **nodes** (entities) and **edges** (typed relationships). The formal apparatus is graph theory; the practical significance is that certain questions are answerable only by traversal. If a document states that a project uses a dataset, and another states that the dataset was used to train a model, then “which model relates to my project?” requires following a path through two nodes and finding no single document that contains the answer. This is a **multi-hop** question, and it is why graph-based retrieval exists.

A MULTI-HOP QUESTION

"Which model is compared on my pollution project's dataset?"



No single passage contains the answer. Retrieval over the graph traverses: project → dataset → model.

The class of question that motivates graph retrieval. SecondBrain's graph holds 350 nodes and 500 edges, built incrementally from ingested material.

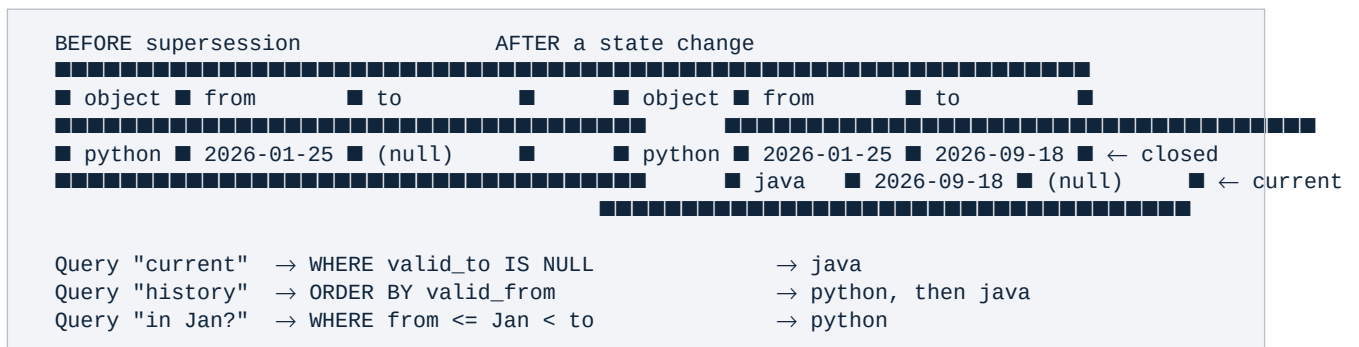
Two design questions arise. First, how are nodes created — by extraction from text, by a fixed ontology, or both? SecondBrain extracts entities during curation and links them, which risks duplicate nodes for the same concept but requires no ontology maintenance. Second, how is the graph used at retrieval time? Here it is used as a **re-ranking signal**: a memory whose concepts are well connected in the graph is more central to the user's interests and ranks higher. That is the third evaluated pipeline.

17. Temporal representation and validity intervals

A conventional database records current state. Updating a row overwrites its previous value, and history is lost. This is correct for most applications and wrong for a personal memory system, which must answer both “what am I working on?” and “what was I working on in January?”

The technique is **validity-interval** (or **bitemporal**) modelling. Every assertion carries a time range during which it holds, written [valid_from, valid_to). The interval is half-open: it includes the start and excludes the

end, so adjacent intervals tile without overlap. A null `valid_to` means “still true”, which is the sentinel for an open assertion.



Validity intervals make the current and the historical question answerable from one store. Deletion would answer the first and destroy the second.

The normalisation defect that defeated the model

A temporal model is only as correct as the normalisation feeding it. In the first implementation, “I am learning Python” was stored under the predicate *learning* and “I am now focusing on Java” under *focus*. Because the predicates differed, neither assertion was recognised as superseding the other, and the graph reported both as current — reproducing precisely the failure the temporal model exists to prevent. The fix was to canonicalise both into a single predicate *current_focus* before applying supersession. The wider lesson: *a data model is only as correct as its normalisation, and the failure is silent.*

18. Truth maintenance and belief revision

Truth maintenance is the problem of keeping a store of beliefs consistent as new information arrives. The general problem is undecidable; practical systems therefore detect specific, tractable classes of inconsistency rather than attempting full consistency.

The classical formulation distinguishes **justification-based** systems, which record why each belief is held and retract beliefs whose justifications are invalidated, from **assumption-based** systems, which track which combinations of assumptions are consistent. Both are more machinery than a personal assistant needs. This project implements a pragmatic subset: detect three specific conflict classes, and apply a policy rather than a full revision.

Conflict class	Detection signal	Policy applied
Temporal supersession	Two assertions share a normalised predicate and differ in object; the earlier is closed	Damp the older by severity; never delete
Lexical negation	Shared topic signature; one text contains a word-boundary negation the other does not	Prefer the later assertion; record the conflict
Value change	Shared topic signature with high salient-word overlap, but different extracted dates or numbers	Prefer the later value; damp the earlier

Table 12 — The three conflict classes and their resolution policies. The third class was absent from the original design and was added after testing showed the first two missed the most common real case.

The resolution policy deserves scrutiny. The obvious response to a contradiction is to remove the outdated item. This resolves the stale-answer problem and destroys the ability to answer historical questions — and both capabilities are legitimate. The system therefore **damps rather than deletes**: a conflict of severity s multiplies the item's ranking weight by $(1 - 0.7s)$, bounded to the range $[0.3, 1.0]$. The outdated item sinks below the current one without leaving the store.

Why damping alone was not enough — and made things worse

The first implementation of damping multiplied the damped score by the memory's *importance*. This promoted memories that were important in the abstract above the memory that actually answered the question, and on several benchmark questions it ranked the correct answer below an irrelevant one — so the proposed method scored worse than its baselines. The fix was to make re-ranking **rank-preserving**: the fused retrieval order supplies a prior of $1/(1+i)$, the damping multipliers adjust it, and importance acts only as a light tie-breaker. The principle generalises: *a re-ranking layer should refine an ordering, not replace it, unless it has strong evidence to do so.*

19. Memory: from cognitive science to software

The term 'memory' is used differently in psychology, neuroscience and computing, and the borrowings between them are often loose. It is worth being precise about which distinctions are load-bearing here.

Distinction	In psychology	Computational analogue in this system
Short-term vs long-term	A small, rapidly decaying working store versus durable storage	The conversation context window versus the persisted memory table and vault
Episodic vs semantic	Memory of specific events versus memory of general facts	Timestamps and session records versus compiled wiki articles and graph entities
Procedural	Memory of how to perform an action	Not modelled. The agents encode procedure; the memory does not learn it.
Consolidation	Stabilisation and integration of memories, associated with sleep	Merging near-duplicate memories into a canonical record with provenance retained
Forgetting	Adaptive decay and interference, mostly passive	Deliberate, user-initiated and enforced at every retrieval surface
Reconsolidation	A recalled memory becomes labile and can be updated	Not modelled. Memories are immutable; new assertions supersede rather than edit.

Table 13 — Cognitive distinctions and what the system does with each. The two deliberate omissions are as informative as the implementations: this is an engineering system, not a cognitive model, and claiming otherwise would be overreach.

The strongest borrowing is consolidation, and the analogy is genuinely useful rather than decorative: a system that stores every observation separately becomes dominated by redundancy in the same way a mind that retained every perception would be overwhelmed. The weakest borrowing is forgetting, where the analogy misleads — human forgetting is passive and lossy, whereas the operation here is deliberate, precise, auditable and reversible. The system does not 'forget' in any cognitive sense; it removes content from retrieval by explicit and recorded instruction.

20. Machine unlearning

Unlearning is the problem of removing the influence of specific data from a system that has already incorporated it. Its prominence is driven by privacy regulation granting a right to erasure, combined with the impracticality of retraining a large model on every such request.

A retrieval-augmented system faces a narrower version of the same problem, and the narrowness is deceptive. Deleting the record from the primary table appears sufficient. It is not, because the content exists in at least five independent places, each of which can return it to a result set:

WHERE A DELETED RECORD CAN SURVIVE

1	full-text search index	a row keyed to the memory id
2	vector store	an embedding, retrievable by similarity
3	knowledge graph	nodes derived from the memory, with incident edges
4	temporal fact store	facts extracted from the memory, still open
5	derived research tables	importance score, conflict records, consolidation membership

Deleting only from the primary table leaves five retrieval surfaces intact.
The measured result: baselines leaked at rate 0.022; the adaptive pipeline, which enforces all five, leaked at 0.000.

The five surfaces. The measured contrast between baselines and the adaptive pipeline is the empirical evidence that this is a real problem and not a theoretical one.

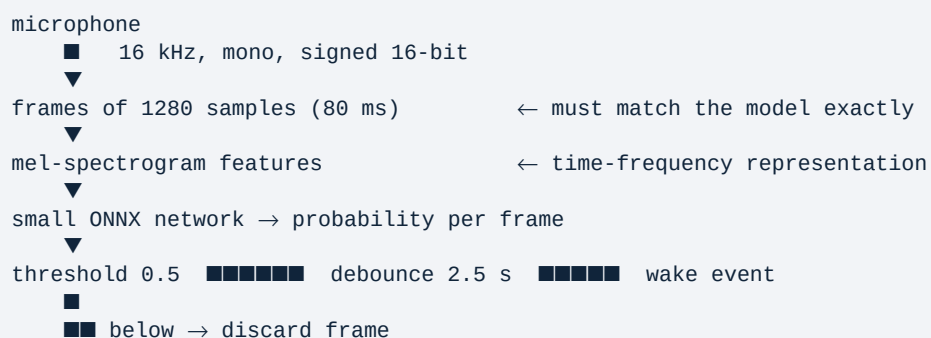
SecondBrain's implementation tombstones the memory — keeping the row with a marker rather than deleting it, so identifiers stay stable and the removal is auditable — and then enforces exclusion at all five surfaces. Restoration re-indexes and re-admits, so the operation is reversible. The finding that *every baseline continued to return explicitly unlearned content* generalises beyond this project: any system that promises deletion while maintaining a derived index has an unlearning hole, and the hole is invisible from the primary table.

21. Signal processing for keyword spotting

A **wake word** system must listen continuously while remaining cheap enough to run indefinitely on a laptop. It is therefore not speech recognition. It is **keyword spotting**: a small model that scores short audio frames and fires when a target phrase is recognised. Transcription happens only after the wake event.

The signal chain is as follows. Audio is sampled at 16 kHz in mono, because speech recognition models generally expect that rate and the frequencies above 8 kHz contribute little to intelligibility. It is divided into fixed frames — here 80 ms, or 1280 samples — because the model expects a fixed input shape, and the frame length must match exactly or the model receives a malformed input. Each frame is scored by a small neural network, typically producing features derived from a spectrogram, and the resulting probability is thresholded.

THE AUDIO PIPELINE



Measured: "Hey Jarvis" 0.9968 | silence 0.0000 | noise 0.0007 | tone 0.0018

The keyword-spotting chain. The measured separation between the positive case and every negative control is what justifies the 0.5 threshold.

Three engineering concerns dominate. **Frame size** must match the model's expectation precisely. **Threshold** trades missed detections against false activations, and should be chosen from the measured score distribution rather than assumed. **Debounce** prevents a single spoken phrase from firing the event repeatedly as its trailing energy decays through the threshold.

Why the negative controls matter more than the positive one

A detector that fires on everything would pass a test that only checks whether it fires on the wake phrase. The informative measurements are silence, broadband noise and a pure tone — all of which scored below 0.002 while the phrase scored 0.9968. That margin is what makes the threshold defensible, and it is why the regression suite asserts the negative cases explicitly.

22. Authentication, authorisation and scopes

Authentication establishes who is making a request; **authorisation** determines what they may do. This system uses both, at two levels, and the two levels are frequently confused.

Level	Mechanism	What it protects
Application access	JWT bearer token, issued at login, attached to every request	The user's own data from anyone else on the machine or network
External service access	OAuth tokens or app passwords, stored encrypted	The user's mail and calendar accounts

Table 14 — Two authentication layers. A defect in the first layer is what made 52 endpoints return 401 while the interface showed empty panels (Volume VI).

OAuth lets a user grant an application limited access without revealing a password. The user authenticates with the provider; the application receives a token. Access is described by **scopes**, and providers tier scopes by sensitivity because the consequences of misuse differ enormously.

SCOPE TIERS AND THEIR COSTS

NON-SENSITIVE	openid, email, profile → self-assessment only; publishing is immediate
SENSITIVE	send mail, modify calendar events → provider verification review
RESTRICTED	read a mailbox, modify a mailbox, full mailbox access → third-party security assessment (CASA) → paid, several hundred dollars annually → annual re-certification → weeks of review

Reading an inbox is a RESTRICTED scope. This single fact determined the project's entire mail strategy (deviation D7).

The scope tiering that made the Google API route uneconomic for a single-user application, and which the author's question about a commercial assistant surfaced.

App passwords are an older mechanism: once two-factor authentication is enabled, the provider issues a long random password valid only for legacy protocols such as IMAP. There is no consent screen, no verification requirement and no expiry. Their security properties differ rather than being strictly worse: they are scoped to a protocol rather than to an API, and they are revoked by the user rather than by a token service.

The read-only guarantee in this project is not a configuration flag but a property of which commands are issued. IMAP distinguishes EXAMINE (open a mailbox without altering it) from SELECT (open it read-write), and BODY.PEEK[] (fetch without setting the read flag) from BODY[] (fetch and mark read). An implementation using the latter would silently mark the user's entire inbox as read the first time it synchronised — a failure the user would notice only indirectly, and one that the test suite therefore asserts against explicitly.

23. Evaluation theory and the design of metrics

A claim that system A is better than system B is meaningless without a measurement, and the choice of measurement determines what can be concluded. This chapter explains the metrics used and, more importantly, why three of them had to be invented.

Metric	Definition	Sensitivity	Purpose
Hit@k	Does a relevant item appear in the top k?	Position-insensitive	Standard recall. Answers 'did we find it?'
MRR	Mean of 1/rank of the first relevant item	Rank-sensitive	Answers 'did we find it first?'
Precision@k	Fraction of returned items that are relevant	Set-based	Measures context efficiency
nDCG	Graded relevance discounted by rank	Rank-sensitive, graded	Not used: this corpus has binary relevance
Stale@1	Is the top-ranked item superseded or forbidden?	Rank-1 specific	Custom. Rank one is what a generator leads with
Leak rate	Fraction of returned items that were explicitly forgotten	Set-based	Custom. Detects unlearning holes
Duplicate rate	Fraction of within-query pairs that are near-identical	Set-based	Custom. Measures wasted context budget

Table 15 — Metrics and their sensitivities. The custom metrics were necessary because the standard ones proved unable to distinguish the systems.

The result that made the custom metrics necessary

The expectation before measuring was that the research layer would improve Hit@5. It did not: all four pipelines reached 1.000. Every system found the relevant memory somewhere in its top five. Had the evaluation stopped there, the honest conclusion would have been that seven components accomplished nothing measurable — and that conclusion would have been wrong. What differed was the *ordering* and the *staleness of what came first*. The general lesson: *a metric that saturates conveys no information, and saturation should be investigated rather than reported*.

Two further methodological points apply. First, **metrics must be computed over the appropriate population**: including abstention and removal questions, which have no correct answer by construction, would depress every system equally and conceal real differences, so Hit@k and MRR are computed over answerable questions only. Second, **reproducibility is a precondition, not a nicety**: two consecutive runs of a non-deterministic harness cannot support a comparison, and this project's benchmark was made deterministic by clearing derived state between runs and by using a deterministic retrieval fallback.

24. Software verification theory

Verification is the discipline of establishing that software does what is claimed. The techniques used here map onto the classical levels, and the project's central methodological lesson concerns the boundary between them.

Level	Technique used here	What it can establish
Static analysis	A linter failing on undefined identifiers and unused symbols	That every referenced name is defined. Cannot establish that defined names are correct.
Static integration checks	Eight checks: imports resolve, icons exist, client methods exist, route modules exist, URLs match, preload surface complete, Python compiles	That the parts agree on names and shapes. Cannot establish that a request succeeds.
Contract checking	Frontend URLs matched against backend routes, including both branches of ternary expressions	That no call targets a non-existent route. Cannot establish that the route works.

Level	Technique used here	What it can establish
Render and mount testing	Every interface route rendered and mounted so effects execute	That components do not crash on construction or on effect. Cannot establish that data is correct.
Functional suites	Eleven backend suites plus eight specialised suites, roughly 400 assertions	That specified behaviours hold under specified conditions.
Live endpoint sweep	Boot the server, authenticate, call every route, report status and timing	That every route answers without a server error under real conditions.

Table 16 — The verification levels used, ordered by what they can establish. This ordering is the report's most important methodological claim.

The lesson that cost the most time

The first error audit used correct and useful techniques. It found the visible crash and one more of the same class, and it produced permanent tooling that has caught defects since. It could not possibly have found the project's most serious defect, because *a login function that raises before comparing a password is syntactically valid code*. No amount of static analysis detects a connection that is missing rather than wrong. Only executing the system and attempting to log in revealed it, in minutes. The general principle: *static analysis finds what is written incorrectly; only execution finds what is written plausibly but connected to nothing*.

25. Local inference and resource constraints

Running a language model locally rather than calling a hosted service changes the design space in ways that are easy to underestimate. The constraint is memory: a model's weights must fit in GPU memory alongside the activations needed for inference.

WHY MODEL SIZE IS BOUNDED HERE

GPU memory available ≈ 6 GB (RTX 3050 Laptop)
weights at 16-bit precision ≈ 2 bytes per parameter
activation and context overhead ≈ 1-2 GB at typical context lengths

∴ practical ceiling ≈ 2-3 billion parameters at 16-bit
∴ a 3B model was selected, and the design assumes it may be absent

QUANTISATION trades precision for size:

16-bit → 8-bit roughly halves memory, small quality loss
8-bit → 4-bit halves again, larger quality loss

The consequence for architecture: every model-dependent feature was made OPTIONAL, and each degrades to a deterministic non-neural fallback.

The resource constraint and its architectural consequence. The ceiling on model size is why the optionality principle is not merely tidiness.

Three practical lessons emerged. First, **token streaming matters more locally than remotely**, because a small model's first token can be slow while subsequent tokens flow quickly; showing text as it arrives makes the latency tolerable. Second, **a configured model may be absent** — a model can be re-pulled under a different tag, or a different machine may have a different set — so the system queries what is actually installed and selects accordingly rather than failing with a 404. Third, and most importantly, **a local model is a dependency that can be unavailable**, and the architecture must therefore treat every model call as fallible.

26. Engineering practice: what the project taught

This final chapter of the theory volume abstracts the practical lessons, because they generalise beyond this codebase.

Principle	Stated plainly	Evidence from this project
A build proves compilation, not execution	Succeeding to build is necessary and nowhere near sufficient.	A missing component import passed every build and crashed the application at runtime. Twice.
A silent failure is worse than a loud one	The cost of a defect is proportional to how long it remains undetected, not to its severity.	Broken authentication produced empty panels rather than errors, and remained undetected through a full audit.
An error code is a claim about causation	Choosing a status code is choosing where the reader will look for the fault.	Reporting a missing record as an upstream failure sent investigation in exactly the wrong direction.
Test the claim, not the code	If a claim cannot be tested, it should not be made.	Writing one assertion per paper claim forced the question 'what would this look like if it were broken?' and found several domain-model defects.
Prefer negative controls	A test that only checks the positive case passes on a system that does nothing.	Silence must not trigger the wake word; unrelated memories must not be duplicates; a forgotten memory must not appear in any retrieval mode.
Prove that the check can fail	A green result is only meaningful if red is possible.	The crash-class defect was deliberately re-introduced to confirm the gate refused it. Doing this also exposed three false positives in the checkers themselves.
Destructive operations must prove ownership	A cleanup routine must be able to demonstrate that what it is deleting is its own.	A test deleted 123 of the author's real memory cards because it could not distinguish its own artefacts from genuine data.
Centralise cross-cutting concerns	Duplicated plumbing drifts, and the drift is invisible.	33 call sites each independently failed to attach an authentication token.
A model is only as good as its normalisation	Correct code over an incorrectly normalised model is silently wrong.	Two facts describing one state were stored under different predicates, so neither superseded the other — reproducing exactly the failure the component existed to prevent.
Instrumentation compounds	The value of verification is realised on every subsequent change, not on the change that motivated it.	Four phases of work were completed in a single day once the fifteen gates existed, because each change could be validated in under a minute.

Table 17 — Ten engineering principles, each with the evidence from this project that produced it. These are the most transferable output of the work.

Volume V — Register of Deviations

A deviation register records every point at which the implementation departed from the plan, with the reasoning. It exists because a project cannot be assessed for judgement without one — only for adherence, and adherence is not the quality that matters. Thirteen deviations are documented.

Deviation D1 — Core framing

Planned	Build a note-taking application with search and tags.
Actual	Abandoned notes entirely; built an autonomous capture and retrieval system.
Reason	A system whose value depends on manual curation will be abandoned, because the user is bad at curation — that is why the problem exists. Moving judgement from human to machine is harder engineering and is the only version that works.

Deviation D2 — Model transport

Planned	Call the local model through its OpenAI-compatible endpoint.
Actual	Call the native local API directly.
Reason	The compatibility layer returned 'model not found' for models that existed, and diagnosing it took longer than it should have. An adapter adds a failure mode without adding capability when talking to your own infrastructure.

Deviation D3 — Image storage

Planned	Keep every captured frame.
Actual	Score frames for information density and retain only the best per information period.
Reason	Storing everything grows without bound and retrieves nothing useful. The criterion for 'best' must be measurable, and text density is.

Deviation D4 — Scraping

Planned	Use paid scraping services for social and web ingestion.
Actual	Built a native stack on free libraries; ingestion fell to 0.98 s.
Reason	Directed by the author: build the capability rather than rent it. The principle was then applied unprompted to mail and calendar, where it avoided a recurring cost (D7).

Deviation D5 — Interface

Planned	A clean dashboard with a side navigation.
Actual	Three successive rebuilds, landing on persistent navigation plus an ambient command centre.
Reason	The requirement could not be specified in advance. Each rebuild eliminated a wrong interpretation: the first showed the visual language was wrong; the second showed that removing navigation makes functionality unfindable.

Deviation D6 — Modality

Planned	Answer questions in the interface.
Actual	Added speech in and out, a transparent floating orb, and proactive spoken announcements.

Reason	Derived from the author's reference videos once analysed for substance rather than appearance: the defining characteristic of the reference assistant is that it speaks first.
--------	--

Deviation D7 — Account connectivity

Planned	Connect accounts through OAuth and publish the application.
Actual	Added a credential-free path using IMAP and calendar feeds, and made it the default.
Reason	Researching the author's question about a commercial assistant revealed that reading an inbox requires a restricted scope, and publishing one requires an annual third-party security assessment costing several hundred dollars. For a single-user application the cost is indefensible, and staying unverified forces weekly reconnection of every account.

Deviation D8 — Invocation

Planned	The user presses a hotkey to summon the assistant.
Actual	Added offline wake-word detection so the assistant can be addressed by name.
Reason	A name is most of what distinguishes an assistant that is present from an application that is opened. Implemented offline so no audio leaves the machine.

Deviation D9 — Order of work

Planned	Write the paper describing the seven planned contributions.
Actual	Audited the claims, found six unimplemented, built them, then wrote from measured results.
Reason	A paper written before the software describes something that does not exist. The audit also changed the paper's content: the predicted result was wrong, so the paper reports the measured result instead.

Deviation D10 — Re-ranking

Planned	Re-rank retrieved memories by their computed importance score.
Actual	Use the retrieval order as a prior and apply damping as an adjustment.
Reason	Sorting by importance was actively harmful, promoting memories important in the abstract above the memory that answered the question. A re-ranker should refine an ordering, not replace it.

Deviation D11 — Conflict classes

Planned	Detect temporal supersession and lexical negation.
Actual	Added numeric and dated value-change detection as a third class.
Reason	Testing showed the first two missed the most common real case — a deadline that moved, expressed in the third person with no negation word and no first-person state claim.

Deviation D12 — Similarity threshold

Planned	Use 0.90 cosine similarity to identify duplicates.
Actual	Set the threshold to 0.78 from the measured distribution.
Reason	Reworded duplicates, the common real case, score around 0.82. A 0.90 threshold missed every one of them. A threshold chosen to make the test pass would have been fitting the method to the evaluation.

Deviation D13 — Reporting the ablation

Planned	Report a per-component ablation table.
Actual	Removed the table and stated in the paper that the factorised ablation is not yet run.
Reason	Some cells had not been independently measured. A table implying measurements that were not taken is worse than an honest gap, and a reviewer would eventually ask.

The two deviations that changed the project's direction

D7 (account connectivity) removed a recurring cost and, more importantly, removed a dependency on a third party's willingness to approve the application. **D9** (order of work) prevented a paper from describing software that did not exist, and produced the project's most interesting finding — that the anticipated improvement was absent while a different, unmeasured property had changed substantially. Both deviations were triggered by a direct question from the author challenging an assumption.

Volume VI — Register of Defects

Twenty-seven significant defects, grouped by cause rather than chronology because the causes cluster and the clustering is the useful information. Five root causes account for almost everything: silent failure, incorrect error semantics, a wrong model of the domain, lifecycle gaps, and unverified process. Each entry records the symptom, the cause, the fix, and the lesson.

Class 1 — Missing or unverified dependency (D1–D4)

Defect	Symptom	Root cause and fix	Lesson
D1 Undefined identifier, twice	Blank screen on the home route; a second instance waiting on another tab	A component was used in markup without being imported. Fixed by adding the imports; a linter rule now fails the build on any undefined identifier.	A successful build proves compilation, not execution.
D2 Undeclared package	A fresh installation crashed on startup	Two packages were imported directly but listed only as transitive dependencies of another package, and resolved locally by accident. Both declared explicitly.	A direct import needs a direct declaration, however it happens to resolve on your machine.
D3 Library version drift	Wake-word detection would have failed silently on the target machine	The model-loading call used an argument name present only in a newer release than the one installed. Loading made version-tolerant, attempting four constructor shapes.	Test against the installed version, not the documented one.
D4 Dependency conflict	A clean installation refused to complete	A linting package declared a peer requirement for a major version contradicting the pinned version. Versions aligned.	A build that works only on the author's machine is not a build.

Table 18 — Dependency defects.

Class 2 — Silent failure (D5–D8)

This class caused more lost time than all others combined, and the reason is structural: a system that fails quietly is misread as working, so the fault is attributed to the wrong component or to nothing at all.

Defect	Symptom	Root cause and fix	Lesson
D5 Authentication had never worked	The interface rendered normally but showed empty data everywhere	A service referenced a configuration object without importing it, so login raised before comparing any password. The frontend swallowed the error, so the application ran with no token and 52 protected endpoints returned 401 — rendered by the interface as empty states. Fixed by adding the import.	The most dangerous failure is the one the interface misrepresents. Found only by running the server and attempting to log in.
D6 Thirty-three unauthenticated calls	Even a correct login would not have authenticated most screens	Screens used a raw browser fetch, which does not attach the token, instead of the application's client. Fixed with a single authenticated helper and a mechanical replacement across 18 files.	Duplicated plumbing drifts, and the drift is invisible in review.

Defect	Symptom	Root cause and fix	Lesson
D7 A test passing while the system was offline	An endpoint reported success while the language model was unreachable	The endpoint always returns success and substitutes canned text when the model is absent; the test checked only the status code. The test now inspects for fallback markers and reports DEGRADED with the actual cause.	A test that cannot fail is not a test.
D8 Deliberate errors re-labelled as faults	Behaviour differed between runs with no error reported	A generic exception handler caught deliberate errors and re-labelled them as server faults, so a missing record reported an internal error with a not-found body. Deliberate exceptions re-raised before the generic handler.	Catch narrowly. A broad handler converts informative errors into uninformative ones.

Table 19 — Silent-failure defects.

Class 3 — Incorrect error semantics (D9–D10)

Defect	Symptom	Root cause and fix	Lesson
D9 A missing record reported as an upstream failure	Six endpoints returned a gateway-error code for a non-existent identifier	The exception types for 'no such account' and 'the provider rejected us' were the same, and both mapped to the upstream-failure code. A distinct type was introduced for the missing-account case, mapped to not-found.	An error code is a claim about where the fault lies. Saying 'upstream' when the request was wrong sends the reader in the wrong direction.
D10 Status depended on ambient configuration	The same request returned different codes in different environments	Routes called the provider before checking whether the account existed, so the response depended on whether the provider was configured. Every account-scoped route now resolves the account locally first.	A response should be a function of the request, not of unrelated global state.

Table 20 — Error-semantics defects.

Class 4 — Wrong model of the domain (D11–D16)

This is the most interesting class. In every case the code was correct and the *understanding* was wrong, and in every case the defect was invisible until measured.

Defect	Symptom	Root cause and fix	Lesson
D11 Temporal supersession silently did nothing	'What am I working on?' returned both the old and the new answer	Two facts describing one state were stored under different predicate names, so neither superseded the other. Predicates are now canonicalised into state classes before supersession.	A data model is only as correct as its normalisation, and the failure is silent. This reproduced exactly the failure the component existed to prevent.
D12 Negation detected by substring	Notes about notes were reported as logical contradictions	Negation was tested with a substring check, so 'not' matched inside 'notes', 'notification' and 'another'. Replaced with word-boundary matching against an explicit pattern list.	Substring matching on natural language is almost always wrong — it is the classic error in this domain.

Defect	Symptom	Root cause and fix	Lesson
D13 The most common contradiction was undetectable	A deadline moving from October to September was missed entirely	Both the temporal extractor and the negation detector keyed on first-person state claims or negation words; a third-person factual value change matches neither. Added dated and numeric value-change detection requiring shared topic signature and high salient-word overlap.	Test the common case, not the designed case.
D14 Topic words stripped as noise	The strongest real contradiction stopped being detected after a precision tightening	The stop-word list excluded 'deadline' and 'project' as filler — precisely the words that make two memories comparable. Restricted to generic filler only.	A stop-word list is a model of the domain, and it can be wrong in ways that silently destroy the signal.
D15 Importance ranking made results worse	The proposed method scored below a baseline	Re-ranking multiplied the damped retrieval score by an abstract importance score, promoting low-relevance high-importance memories above the correct answer. Made rank-preserving with importance as a light tie-breaker.	A re-ranker should refine an ordering, not replace it.
D16 Transparency required three simultaneous changes	A dark rectangle behind the floating assistant	Window transparency on Windows requires the window flags, the document-root styling and the body background to agree. Changing any one alone has no effect.	When a visual defect survives a fix, the fix is usually correct but incomplete across layers.

Table 21 — Domain-model defects. Every one was found by measurement rather than by inspection.

Class 5 — Lifecycle and cleanup (D17–D21)

Defect	Symptom	Root cause and fix	Lesson
D17 Cleanup deleted the last reference	Broken images in the live view	The pruning process deleted raw frames that a preview still referenced. A dedicated buffer was excluded from pruning.	Never let cleanup delete the last reference to something a view depends on.
D18 Orphaned derived rows	A test crashed on a record whose parent no longer existed	Deleting a memory left its score row behind, inflating counts while never appearing in the joined listing. Orphan pruning added.	Derived data needs a lifecycle, not merely a creation path.
D19 The microphone never re-armed	The assistant answered one question and then stopped	The browser's speech recogniser ends its session by itself and no handler restarted it; the summon gesture only spoke a greeting and never restarted recognition. Re-arm added in the session-end handler with backoff.	An event-driven resource that ends itself needs an explicit restart path.
D20 Two further ways the assistant could become unresponsive	Not observed; found by writing the tests	A turn the model never answers leaves the microphone paused indefinitely; and the text-to-speech completion event is not guaranteed in Electron when the window is hidden. Two watchdogs added.	Write the test for the failure you have not seen yet. It found both before the author did.
D21 A wake event could hide the assistant	Not observed; found by reading the code	The wake handler called a function that toggles window visibility rather than one that shows it, so waking an already-visible assistant would hide it. A non-toggling reveal path added.	'Show' and 'toggle' are different operations; conflating them produces inverted behaviour.

Table 22 — Lifecycle and cleanup defects. Two of the five were found by tests written for hypothetical failures rather than by observing a real one.

Class 6 — Process and data safety (D22–D27)

Defect	Symptom	Root cause and fix	Lesson
D22 Benchmark not reproducible	Results changed between consecutive runs	Derived research state from a previous run — temporal facts, conflicts, consolidations, tombstones — perturbed the following run. Derived state cleared at the start of each run; determinism now asserted.	A benchmark that is not deterministic cannot support a claim made from it.
D23 The test deleted real user data	123 of the author's genuine memory cards were removed	The mail-ingestion test registered the author's real address, so its generated artefacts were indistinguishable from genuine ones and its cleanup could not tell them apart. Files restored from version control; the test now uses a non-existent address and its cleanup refuses to delete anything not carrying the test marker, reporting what it protected.	A destructive operation must be able to prove that what it is deleting is its own. This is the most serious defect in the project.
D24 The verification tool crashed on the target machine	A test failed on the user's machine but passed in development	A browser global that was assignable in the older runtime is a read-only accessor in the newer runtime installed on the user's machine. Globals now installed via a descriptor-based helper and verified on both versions.	A check that fails the same way as the product is worse than no check.
D25 The contract checker produced false positives	Correct endpoints reported as missing	The URL extractor and normaliser handled template expressions and query strings in the wrong order, truncating a URL at a question mark inside a ternary. Order corrected and ternary branches now expanded into separate candidates.	A checker that cries wolf gets ignored. The mislabelling of a timeout as a crash was therefore treated as a bug in its own right.
D26 A stage ran before its dependencies existed	Scripts failed with an error naming an unrelated class	Model relationships were being resolved before all model modules had been imported. A bootstrap import was added at the top of the dependent package.	Import order is a real dependency. Make the requirement explicit rather than relying on the order a file happens to run.
D27 Output correct in the log, wrong on the page	A generated document reported success but the layout was broken	The build log reported success because the document was produced; the layout defect was only visible on inspection. Fixed by rendering each page to an image and examining it.	Verify the artefact, not the exit code. This is the same lesson as D1 at a different layer.

Table 23 — Process and data-safety defects.

What the register as a whole indicates

The distribution is informative. Dependency defects were eliminated once the verification system existed and have not recurred. Silent-failure defects cluster early, before the live sweep existed. Domain-model defects were found exclusively by measurement, never by inspection. Lifecycle defects were found by tests written for hypothetical failures. And the single most serious defect was caused not by the application but by the verification process itself — which is why its fix is structural rather than local.

Volume VII — The Research Contribution

27. The research question

The question the paper addresses is: *how can a personal AI continuously update, consolidate, prioritise and temporally reason over a user's evolving knowledge without accumulating redundant, outdated or contradictory memories?*

It is worth being explicit about why this is a research question rather than an engineering task. The engineering question would be 'can we store and retrieve a person's information?', and the answer is trivially yes. The research question concerns a property that emerges only over time and only under contradiction: what should a system believe when its own records disagree, and how should that belief affect retrieval? That question has no obvious answer and no standard measurement.

28. The seven components

#	Component	Function	Defining design decision
C1	Adaptive Memory Scoring	Eight-term importance score; all terms persisted	Auditable rather than opaque: any score can be explained term by term
C2	Temporal Personal Knowledge Graph	Triples with validity intervals and state supersession	Close facts rather than delete them, so history remains queryable
C3	Contradiction-Aware Retrieval	Three conflict classes detected; rank damping applied	Damp rather than remove, preserving current and historical answers
C4	Memory Consolidation	Near-duplicate clustering with provenance retention	Threshold calibrated from the measured distribution, not assumed
C5	Knowledge-Gap Detection	Exposure versus depth, weighted by graph connectivity	Connectivity factor prevents flagging well-understood concepts
C6	Retrieval-Enforced Forgetting	Exclusion enforced at five independent surfaces	Enforcement at retrieval, not deletion, because derived stores survive deletion
C7	PersonalBrain-Bench	Synthetic corpus with six question categories	Synthetic by design, so the experiment is reproducible and contains no private data

Table 24 — The seven contributions and the decision that defines each.

29. Benchmark design

Existing retrieval benchmarks do not contain contradictory, superseded or deliberately forgotten personal memories, so they cannot expose the failures this work targets. The benchmark was therefore constructed specifically to isolate them.

Category	Questions	What it tests	Correct behaviour
Factual	3	Ordinary recall	Retrieve the stated fact
Temporal	2	Which of two competing states is current	Retrieve the newer; do not retrieve the superseded one
Contradiction	2	Which of two dated values is authoritative	Retrieve the newer value; damp the older
Multi-hop	1	Synthesis across linked memories	Retrieve the whole chain
Duplicate	1	Whether redundant copies fill the budget	Retrieve one representative; collapse the rest
Forgetting	1	Whether removed content stays removed	Retrieve nothing relating to it

Category	Questions	What it tests	Correct behaviour
Gap	1	Whether the system distinguishes exposure from depth	Report the concept as a gap, not as knowledge

Table 25 — PersonalBrain-Bench question categories. The last two have no correct answer to retrieve by construction, which is why Hit@k and MRR are computed over answerable questions only.

The corpus contains fifteen memories spanning email, notes, code, web captures and screen text, with timestamps distributed across a 240-day window so that recency and supersession are exercised rather than simulated. Synthetic construction is a deliberate methodological choice: a real personal store would make the experiment unreproducible and would place private data in a publication.

30. Results

Pipeline	Hit@5	MRR	Stale@1	Leak	Duplicate	Latency
Vanilla dense	1.000	0.833	0.364	0.022	0.022	33 ms
Hybrid	1.000	0.944	0.091	0.022	0.022	29 ms
Graph-augmented	1.000	0.944	0.091	0.022	0.022	37 ms
Adaptive (proposed)	1.000	1.000	0.000	0.000	0.000	80 ms

Table 26 — Comparative results ($k = 5$, 11 questions, 15 memories). Two consecutive runs are byte-identical.

Question category	Vanilla MRR	Hybrid MRR	Adaptive MRR	Change
Factual	1.000	1.000	1.000	—
Temporal	0.750	0.750	1.000	+0.250
Contradiction	0.500	1.000	1.000	—
Multi-hop	1.000	1.000	1.000	—
Duplicate	1.000	1.000	1.000	—

Table 27 — MRR by category. The improvements appear exactly where the architecture predicts them and nowhere else.

31. Interpretation

The headline finding is negative, and that is the contribution. Hit@5 is 1.000 for every pipeline including the simplest baseline. Every system retrieves the relevant memory somewhere in its top five. Had the evaluation used only recall, the conclusion would have been that seven research components accomplished nothing measurable — and that conclusion would have been wrong.

What differs is the ordering and the staleness of what comes first. The proposed pipeline raises mean reciprocal rank from 0.833 to 1.000 and reduces the rate of leading with superseded content from 36.4% to 0.0%. Because a language model consumes the ordered context and leads with what it reads first, staleness at rank one is what produces an answer that is confidently wrong. Critically, every retrieved passage remains faithful to its source, so no faithfulness metric would flag it — the system is hallucination-free and incorrect.

The category breakdown localises the gains. Factual, multi-hop and duplicate questions are unchanged at 1.000, which is a desirable property: it means the added machinery is a targeted change rather than a general-purpose alteration to retrieval. Temporal questions improve from 0.750 to 1.000 once predicate canonicalisation makes supersession fire. Contradiction questions are where the vanilla baseline is weakest, consistent with the motivating scenario.

The forgetting result is the sharpest. The baselines do not merely score lower; they *continue to return content the user explicitly asked to remove*. The adaptive pipeline returns it zero times. This is evidence that removal from a retrieval system is a property of the retrieval layer and is not obtained by deleting a row from the primary table.

32. Limitations, stated plainly

- **The benchmark is synthetic and small.** Fifteen memories, eleven questions, six categories. The categories isolate specific behaviours; they do not represent the distribution of real personal data. Absolute values should not be extrapolated.
- **Extraction is rule-based and conservative.** It will miss phrased contradictions that do not match its patterns. This trades recall for precision deliberately — a wrong fact in a personal graph is worse than a missing one — but the trade is real.
- **Temporal handling uses UTC-normalised timestamps** without full timezone or partial-interval reasoning, which would matter for events spanning midnight across zones.
- **The evaluation covers retrieval, not generation.** Whether higher MRR and lower stale rate translate into fewer incorrect answers in generated prose is not measured here. This is the natural next experiment and the most significant gap.
- **The ablation is not factorised.** The per-category results localise the effect, but an independent run isolating each of the five post-retrieval stages has not been performed. This is stated in the paper rather than inferred from the aggregate.

The most defensible single sentence in the work

'Every baseline continues to return explicitly unlearned content.' This is a negative result about the baselines. It is measured, reproducible, and not obvious in advance. A contribution does not have to be a large positive number: a precise demonstration that an apparently simple property is not automatic is equally publishable, provided the scope is stated honestly.

Volume VIII — Engineering Practice

33. The verification system

Fifteen gates, run by a single command. The design principle is that each gate must be capable of failing: a check that cannot fail is decoration, and three of the defects in Volume VI were found by examining a passing gate and discovering it was untestable.

Gate	Method	Catches
1 Static analysis	Linters configured to fail on any warning	Undefined identifiers, unused symbols, hook violations
2 Environment health	Eight static integration checks	Imports that do not resolve, icons that do not exist, client methods without definitions, route modules missing, URL mismatches, incomplete preload surface, Python that does not compile
3 API contract	Every interface URL matched against every backend route	Calls targeting non-existent routes, including both branches of ternary expressions
4 Route render and mount	Render then mount every route so effects execute	Component crashes, effect failures, empty renders, broken connected states
5 Production build	Bundle the interface	Compilation failures
6 Backend functional suites	Eleven live suites	Regressions in authentication, capture, graph, vault, wiki, briefing, deliverables, semantic search, OCR, voice
7 Google integration	Fifty checks against a fake provider	Consent URL shape, scope correctness, token exchange, refresh, revoked-token handling, token readability on disk
8 Mail and calendar	Fifty-six checks against a fake IMAP server and a real HTTP fetch	Protocol correctness, iCalendar feature coverage, credential encryption
9 Mail ingestion	Forty-seven checks	That the pipeline can actually answer from ingested mail — the assertion that matters most
10 Orb voice loop	Twenty-two checks driving a fake speech engine	Microphone re-arm, watchdog behaviour, transcript transmission policy
11 Wake word and proactive voice	Fifty-five checks	Policy gates, quiet hours, cooldown, dedupe, and the real model against real audio
12 Hands-free wiring	Ten checks	That wake and speech events actually reach the interface
13 Research layer	Sixty-eight checks, one per claim	Each stated contribution and, crucially, the negative controls
14 Live endpoint sweep	Boot, authenticate, call every route	Server errors under real conditions; reports status and timing for each route
15 Browser smoke (optional)	Load every route in a real browser	End-to-end failures that only appear in a real rendering engine

Table 28 — The fifteen verification gates.

34. Data safety

The project's most serious defect was caused by the verification process rather than by the application: a test that registered the author's real address could not distinguish its own artefacts from genuine data, and deleted 123 real memory cards. The fix was structural rather than local, and the principle it produced is now applied to every destructive operation:

THE SAFETY RULE

A destructive operation must be able to prove that what it is deleting is its own.

Applied here:

- tests use non-existent identifiers (example.invalid domains)
- generated artefacts carry a marker identifying their producer
- cleanup REFUSES to delete anything lacking that marker
- the refusal is reported, so the protection is visible rather than silent

Output from a run:

```
"(cleaned up 4 test vault cards, 4 test wiki articles, restored graph +
wiki index)"
"PROTECTED 123 real vault card(s) and 93 real wiki article(s) – they
did not carry the test marker"
```

Before: the operation succeeded and destroyed real data.

After: the operation succeeds and reports what it protected.

The safety rule and its concrete enforcement. The second line of output is the evidence that the guard works, and it is printed on every run.

35. The development process itself

The working pattern that emerged over the project is worth recording, because it was not planned and it worked well.

Practice	What it means	Why it worked
Reconnaissance before writing	Read the existing code and data shapes before designing a change	Prevented several changes that would have conflicted with existing behaviour
Build the measurement first	For research claims, write the assertion before or alongside the implementation	Several defects were found because writing the assertion forced the question 'what would this look like if it were broken?'
Verify the artefact, not the signal	Render the PDF and look at it; open the page and look at it	Caught two defects where the build reported success and the output was wrong
Report the negative result	State what was measured even when it contradicts the expectation	Produced the project's most interesting finding and its most defensible claim
Document the incident	Record the serious defects in full, including the ones the author would rather omit	A defect register that omits the worst defect is not a defect register

Table 29 — The development practices that emerged, none of which were planned in advance.

Appendix A — Project Statistics

Metric	Value
Total commits	231
Substantive commits	69
Automated vault synchronisations	162
Python source files / lines	165 files, 17,149 lines
Interface files / lines	55 files, 12,449 lines
Backend route modules	21
Backend module directories	44
Specialised agents	10
API endpoints / paths	162 endpoints across 150 paths
Route groups	19
Database tables	49
Interface routes	19
Verification gates	15
Test suites	8 (5 backend, 2 interface, 1 research)
Research assertions	68
Documented defects	27
Documented deviations	13
Knowledge graph	350 nodes, 500 edges
Wiki articles compiled	93
Memory cards from real mail	123

Table 30 — Project metrics at the time of writing. Every figure was read from the repository or from measured output.

Appendix B — Timeline

AUG 2026	Phase 0	Foundations: backend, schema, auth, capture scaffolding
17 SEP	Phase 1	Curation, hero images, vault, wiki compiler, 3D visualisation
18 SEP	Phase 2	Free native scrapers, insight collisions, spoken briefing, deliverable generation, ambient overlay
21 SEP	Phase 3	Three interface rebuilds; fifteen-tab shell; transparent orb
22 SEP am	Phase 4	Two full error audits; fifteen verification gates created
22 SEP	Phase 5	OAuth, then IMAP + iCalendar; mail ingestion pipeline
22 SEP	Phase 6	Wake word, proactive voice, orb voice-loop repair
22 SEP	Phase 7	Research layer: seven contributions, benchmark, IEEE paper
Gates added	0 → 15	Defects found and fixed 27
Deviations	13	Research assertions 68
Phases 4-7 all occurred on one day, which was possible only because		
Phase 4 produced the means to verify a change in under a minute.		

The build at a glance.

Appendix C — Glossary

Agent	A component with a single responsibility, its own failure mode, and no knowledge of the others' internals.
App password	A long random credential issued for legacy protocols after two-factor authentication is enabled. No consent screen, no expiry.
BM25	A classical probabilistic ranking function using term-frequency saturation and length normalisation.
CASA	Cloud Application Security Assessment. A paid annual third-party audit required by a major provider for applications publishing restricted scopes.
Chunk	A passage small enough to embed meaningfully and store as one retrieval unit.
Consolidation	Merging near-duplicate memories into one canonical record while retaining the identifiers of everything merged.
Context window	The bounded amount of text a language model can attend to at once. Makes retrieval a budget-allocation problem.
Cosine similarity	The cosine of the angle between two vectors; the standard measure of semantic similarity. Ignores magnitude.
Damping	Multiplying a retrieved item's ranking weight to demote it without removing it.
Debounce	Suppressing repeated triggering of an event within a short window, so one utterance fires once.
Embedding	A fixed-length numeric vector representing text, positioned so that similar meanings are nearby.
Exposure vs depth	Having encountered a concept versus having understood it.
Hero image	The single representative frame elected from an information period, kept instead of every screenshot.
Hit@k	Whether a relevant item appeared among the top k results. Position-insensitive.
IMAP	Internet Message Access Protocol. The mechanism desktop mail clients use to read mailboxes. EXAMINE and BODY.PEEK preserve read state.
iCalendar	A text format for calendar data. Providers publish a private address per calendar requiring no authentication.
IDF	Inverse document frequency. Weights a term by how rare it is across a corpus.
JWT	JSON Web Token. A signed token carrying a claim about an authenticated identity.
Knowledge graph	Entities as nodes and typed relationships as edges. Supports multi-hop questions that similarity search cannot answer.
Leak rate	Fraction of returned results that were explicitly forgotten. Custom metric.
MRR	Mean reciprocal rank. Rank-sensitive: distinguishes finding the answer first from finding it fifth.
Multi-hop	A question whose answer requires traversing links between separate records.
OAuth	A protocol allowing an application limited access to an account without receiving the password.
ONNX	An open format for neural network models, used here for the wake-word model.
Quantisation	Reducing the numeric precision of model weights to lower memory use, at some cost in quality.
RAG	Retrieval-augmented generation. Retrieving relevant passages and passing them to a language model along with the question.
Scope	A named permission bundle in OAuth. Providers tier scopes by sensitivity.
Stale@1	Whether the top-ranked result is outdated. Custom metric; rank one is what a generator leads with.
Supersession	Closing an older assertion when a newer one replaces it, rather than deleting it.

Agent	A component with a single responsibility, its own failure mode, and no knowledge of the others' internals.
TF-IDF	Term frequency times inverse document frequency. A classical lexical weighting scheme requiring no model.
Truth maintenance	Keeping a store of beliefs consistent as new information arrives.
Unlearning	Removing the influence of specific data from a system that has already incorporated it.
Validity interval	The half-open time range during which an assertion holds. A null end means still true.
Vector store	An index optimised for nearest-neighbour search over embeddings.
Wake word	An always-listening keyword spotter that fires when a target phrase is recognised. Not full speech recognition.

Appendix D — Design Principles, Collected

1	Optionality	no dependency is load-bearing
2	Additivity	extend alongside, never rewrite
3	Testability	each component exercisable in isolation
4	Honest degradation	say what is unavailable; never substitute plausible text
5	Prove the check fails	a green result must be capable of being red
6	Test the claim	if a claim cannot be tested, do not make it
7	Negative controls	a suite of positive cases passes on a system that does nothing
8	Narrow catches	broad handlers convert informative errors into silent ones
9	Correct error codes	a status code is a claim about where the fault lies
10	Deterministic inputs	a non-deterministic harness cannot support a comparison
11	Normalise the model	correct code over a wrong model is silently wrong
12	Calibrate thresholds	a similarity threshold belongs to a data distribution
13	Refine, do not replace	a re-ranker should adjust an ordering, not impose one
14	Cleanup proves ownership	destructive operations must identify their own artefacts
15	Verify the artefact	inspect the output, not the exit code
16	Deterministic memory	an event-driven resource that ends itself needs a restart path
17	Single source of policy	the backend decides; the interface renders
18	Own rather than rent	prefer a one-off effort to a recurring cost

The principles extracted across the project. Items 1–4 were adopted at the start; the remainder were learned from the defects in Volume VI.